

Winter 2026 DSC 240: Homework 2

Zeyu Bian

Update: January 28, 2026

Academic Integrity

Did you receive any help whatsoever from anyone in solving this assignment?

No

Did you give any help whatsoever to anyone in solving this assignment?

No

Assignment 1 (15 points) The Hinge loss for a linear classifier is $\ell(w^T x, y) := \max\{0, 1 - yw^T x\}$. The shifted Hinge loss (aka Perceptron loss) for a linear classifier is $\ell(w^T x, y) := \max\{0, -yw^T x\}$.

(a) Calculate the gradient of hinge loss with respect to w . (you can ignore the case when $w^T x = 1$)

(b) Calculate the gradient of Perceptron loss w.r.t w (You may ignore the case when $w^T x = 0$)

(c) What is the main difference in the SGD algorithm with the two losses above?

Proof. (a) Let $z = w^T x$ Ignoring the boundary case $yz = 1$:

$$\nabla_w \ell(w^T x, y) = \begin{cases} -yx, & yz < 1, \\ 0, & yz > 1, \end{cases} \quad \text{so} \quad \nabla_w \ell(w^T x, y) = -yx \mathbf{1}[yw^T x < 1].$$

(b) Ignoring the boundary case $yz = 0$:

$$\nabla_w \ell(w^T x, y) = \begin{cases} -yx, & yz < 0, \\ 0, & yz > 0, \end{cases} \quad \text{so} \quad \nabla_w \ell(w^T x, y) = -yx \mathbf{1}[yw^T x < 0].$$

(c) With hinge loss, SGD updates whenever an example has margin $yw^\top x < 1$, so it also updates on correctly-classified points that are “too close” to the decision boundary (encouraging a margin). With perceptron loss, SGD updates only on misclassified points ($yw^\top x < 0$), and makes no update once a point is classified correctly.

Assignment 2 (20 points) Figure 1 shows training data with two features, with each example labeled as being in the positive (filled-in points) or negative (open points) class. Proposed linear classifier functions (C1 through C6) are shown as dotted lines, each one indicating a different classifier for this data. Each classifier classifies points to the upper-right of its dotted line as positive and points to the lower-left of its dotted line as negative.

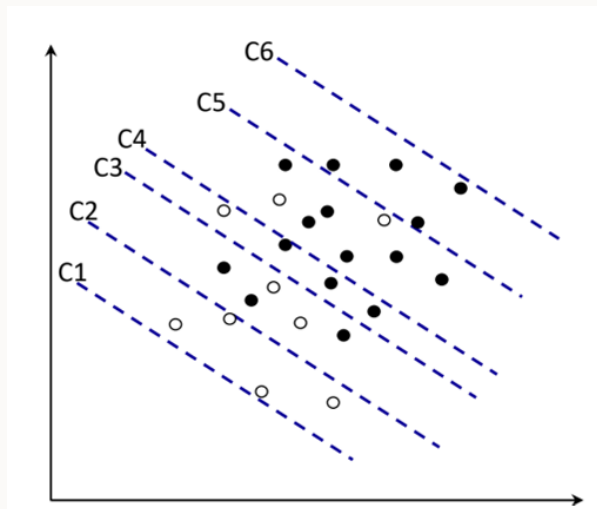


Figure 1: Training data with two features, with each example labeled as being in the positive (filled-in points) or negative (open points) class.

- (a) Draw the ROC plot and label the classifiers on the plot.
- (b) Which classifiers have the highest and lowest accuracy?
- (c) Which classifiers have the highest and lowest precision?
- (d) What are F1 scores for classifiers C1-C6?

Proof. From Figure 1, there are $P = 16$ positive (filled) points and $N = 9$ negative (open) points. Each classifier predicts the region upper-right of its dotted line as positive. As the line shifts from C6 (most strict) down to C1 (least strict), the number of predicted positives increases, tracing out the ROC curve.

- (a)

Confusion matrices and metrics (TPR = recall)

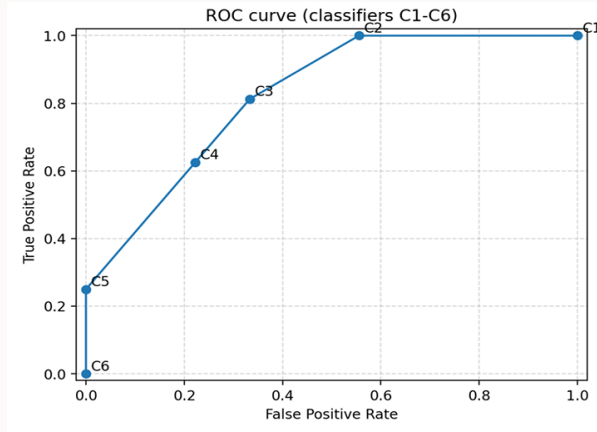


Figure 2: ROC plot

Classifier	TP	FP	TN	FN	TPR	FPR	Accuracy	Precision	F1
C1	16	9	0	0	1.0000	1.0000	0.6400	0.6400	0.7805
C2	16	5	4	0	1.0000	0.5556	0.8000	0.7619	0.8649
C3	13	3	6	3	0.8125	0.3333	0.7600	0.8125	0.8125
C4	10	2	7	6	0.6250	0.2222	0.6800	0.8333	0.7143
C5	4	0	9	12	0.2500	0.0000	0.5200	1.0000	0.4000
C6	0	0	9	16	0.0000	0.0000	0.3600	0.0000	0.0000

b Highest accuracy: C2 (0.80). Lowest accuracy: C6 (0.36).

(c) Highest precision: C5 (1.00). Lowest precision (among predictors with ≥ 1 positive): C1 (0.64).

(d) See the above table.

Assignment 3 (15 points) The decision boundary for a binary classifier that takes an input with three features is the plane defined by the equation $3x_1 + 2x_2 + 4x_3 = 18$, where the features are x_1, x_2 and x_3 . The plane separates the positive predictions ($3x_1 + 2x_2 + 4x_3 \geq 18$) from the negative predictions ($3x_1 + 2x_2 + 4x_3 < 18$).

A dataset set S consists of the following labeled examples:

- $(2, 2, 3)+$ (positive class)
- $(3, 3, 2)+$
- $(1, 2, 3)+$
- $(1, 4, 1)+$
- $(4, 4, 4)+$
- $(2, 2, 2)+$
- $(3, 3, 1)-$ (negative class)

- $(1, 1, 1)$ -
- $(3, 2, 2)$ -
- $(0, 4, 2)$ -
- $(4, 0, 0)$ -
- $(0, 0, 3)$ -

For each data point, provide:

(a) Its margin (distance from the decision boundary)

(b) Its 0-1 loss

(c) Its hinge loss

(d) Its squared loss

Proof. Let $w = (3, 2, 4)$ and define the signed score

$$f(x) = w \cdot x - 18.$$

The classifier predicts $+1$ if $f(x) \geq 0$, else -1 . Also,

$$\|w\| = \sqrt{3^2 + 2^2 + 4^2} = \sqrt{29}.$$

Geometric margin (signed distance) is

$$m(x, y) = \frac{yf(x)}{\|w\|},$$

and the (unsigned) distance to the plane is $\frac{|f(x)|}{\|w\|}$. The losses:

- 0-1 loss: $\mathbf{1}[y \neq \hat{y}]$.
- Hinge loss: $\max(0, 1 - yf(x))$.
- Squared loss (as used in this write-up): $(1 - yf(x))^2$.

x	y	$f(x) = w \cdot x - 18$	margin $yf/\ w\ $	dist $ f /\ w\ $	0-1	hinge	squared
(2, 2, 3)	+1	4.0	0.7428	0.7428	0	0.0	9.0
(3, 3, 2)	+1	5.0	0.9285	0.9285	0	0.0	16.0
(1, 2, 3)	+1	1.0	0.1857	0.1857	0	0.0	0.0
(1, 4, 1)	+1	-3.0	-0.5571	0.5571	1	4.0	16.0
(4, 4, 4)	+1	18.0	3.3425	3.3425	0	0.0	289.0
(2, 2, 2)	+1	0.0	0.0000	0.0000	0	1.0	1.0
(3, 3, 1)	-1	1.0	-0.1857	0.1857	1	2.0	4.0
(1, 1, 1)	-1	-9.0	1.6713	1.6713	0	0.0	64.0
(3, 2, 2)	-1	3.0	-0.5571	0.5571	1	4.0	16.0
(0, 4, 2)	-1	-2.0	0.3714	0.3714	0	0.0	1.0
(4, 0, 0)	-1	-6.0	1.1142	1.1142	0	0.0	25.0
(0, 0, 3)	-1	-6.0	1.1142	1.1142	0	0.0	25.0

Assignment 4 (10 points) Recall the objective function for linear regression can be expressed as

$$E(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2.$$

Minimizing this function with respect to $\mathbf{w}$ leads to the optimal $\mathbf{w}^*$ as $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. This solution holds only when $\mathbf{X}^T \mathbf{X}$ is nonsingular (i.e., full rank or invertible). To overcome this problem, the following objective function is commonly minimized instead:

$$E_2(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2,$$

where $\lambda > 0$ is a user-specified parameter. Please do the following:

(a) (5 points) Derive the optimal $\mathbf{w}^*$ that minimizes $E_2(\mathbf{w})$.

(b) (5 points) Explain how this new objective function can overcome the singularity problem of $\mathbf{X}^T \mathbf{X}$. [Hint: Think about positive definite and positive semi-definite definitions.]

Proof. (a) We minimize

$$E_2(w) = \frac{1}{N} \|Xw - y\|^2 + \lambda \|w\|^2, \quad \lambda > 0.$$

Compute the gradient and set it to zero:

$$\nabla_w E_2(w) = \frac{2}{N} X^T (Xw - y) + 2\lambda w = 0.$$

Rearrange:

$$X^\top X w - X^\top y + N\lambda w = 0 \quad \Rightarrow \quad (X^\top X + N\lambda I)w = X^\top y.$$

Thus,

$$w^* = (X^\top X + N\lambda I)^{-1} X^\top y.$$

(b) $X^\top X$ is always symmetric positive semidefinite, and can be singular if X is not full rank. Adding $N\lambda I$ makes the matrix strictly positive definite: for any nonzero vector v ,

$$v^\top (X^\top X + N\lambda I) v = v^\top X^\top X v + N\lambda v^\top v = \|Xv\|^2 + N\lambda \|v\|^2 > 0.$$

Therefore $X^\top X + N\lambda I$ is positive definite and hence invertible, so the closed-form solution exists even when $X^\top X$ is singular.

Assignment 5 (20 points) In a linear regression task, implement a gradient solver and a stochastic gradient solver for (a) least squares regression (*lsqr*) and (b) ridge regression (least squares with ℓ_2 norm regularization) the given dataset files *data_X_Q5Q6.npy* and *data_y_Q5Q6.npy*. For both *lsqr* and *ridge*, compare the result models from the two optimizers (gradient descent and stochastic gradient descent) and the closed-form solutions (where we set gradient to zero and solved the equation), using the ℓ_2 -norm distance to measure the difference between the two solutions. Please refer to the Appendix section for instructions on how to load the dataset files in python.

Deliverables: Code: Your implementation of gradient solver, stochastic gradient solver, and closed form solver. Your code then load the given dataset(data-.npy), and your code that computes solutions using 3 solvers. Finally, your code that compares weights and reports ℓ_2 -norm distance for each pair. Repeat this whole process with and without regularization.*

PDF: Result of the ℓ_2 -norm distance of weights between 3 comparisons: GD vs SGD, GD vs closed-form, and SGD vs closed-form. Repeat this whole process with and without regularization.

Hyper-parameters and Dataset: Load the data to use in the questions from the directory. For regularization parameters, we suggest $1e-6$.

Proof.

```

=== Q5: Least squares (lambda=0) ===
L2(GD, SGD)      = 7.343713654845933
L2(GD, ClosedForm) = 1.6579744286556386
L2(SGD, ClosedForm) = 7.078054064697164

=== Q5: Ridge (lambda=1.0e-06) ===
L2(GD, SGD)      = 7.337665115077722
L2(GD, ClosedForm) = 1.651386027120106
L2(SGD, ClosedForm) = 7.073708840926423

```

Figure 3: L2 distance

Assignment 6 (10 points) Use the given dataset files `data_X_Q5Q6.npy` and `data_y_Q5Q6.npy` to compute the objective functions (`lsqr` and `ridge`, respectively). Plot the learning curve (x -axis: iteration number, y -axis: objective function). Note that when computing the objective function, $E(w(t))$, you should use all the data points, for both gradient descent and stochastic gradient descent. Please refer to the Appendix section for instructions on how to load the dataset files in python. Deliverables: Code: Using previous implemented solvers and given datasets to generate the curves (with and without regularization; gradient descent and stochastic gradient descent). PDF: The result as curves.

Proof. Results:

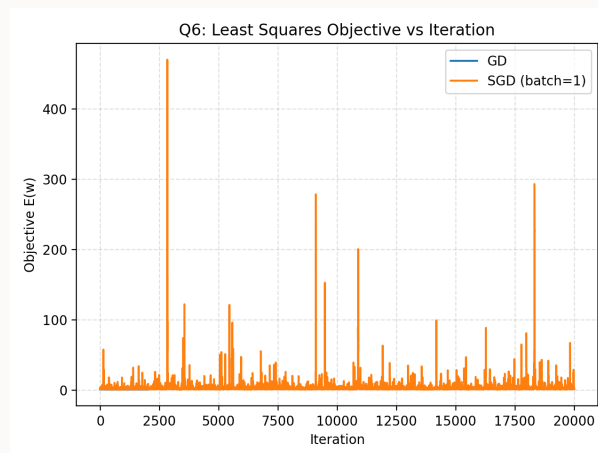


Figure 4: lsqr_gd_vs_sgd

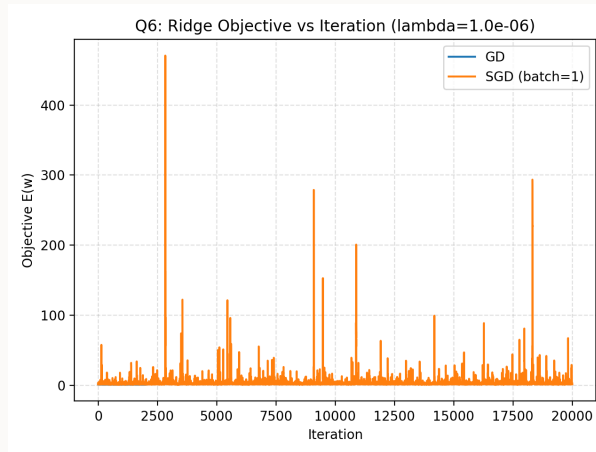


Figure 5: ridge_gd_vs_sgd

Assignment 7 (10 points) Previously, stochastic gradient descent we introduced uses 1 data point in each iteration to estimate the gradient. In fact, the stochastic gradient descent can use a batch of data points to estimate/compute the gradient to accelerate the convergence speed. Implement batched version gradient descent for ridge regression. For this question, use the corresponding dataset files `data_X_Q7.npy` and `data_y_Q7.npy`. Plot the learning curve (again as in Q6, this is objective function against number of iterations) of stochastic gradient descent with batch size of $[1, 10, 100]$. Please refer to the Appendix section for instructions on how to load the dataset files in python. Deliverables: Code: Stochastic gradient descent with different batch sizes. PDF: 3 Convergence graphs with different batch sizes.

Proof. Convergence graph:

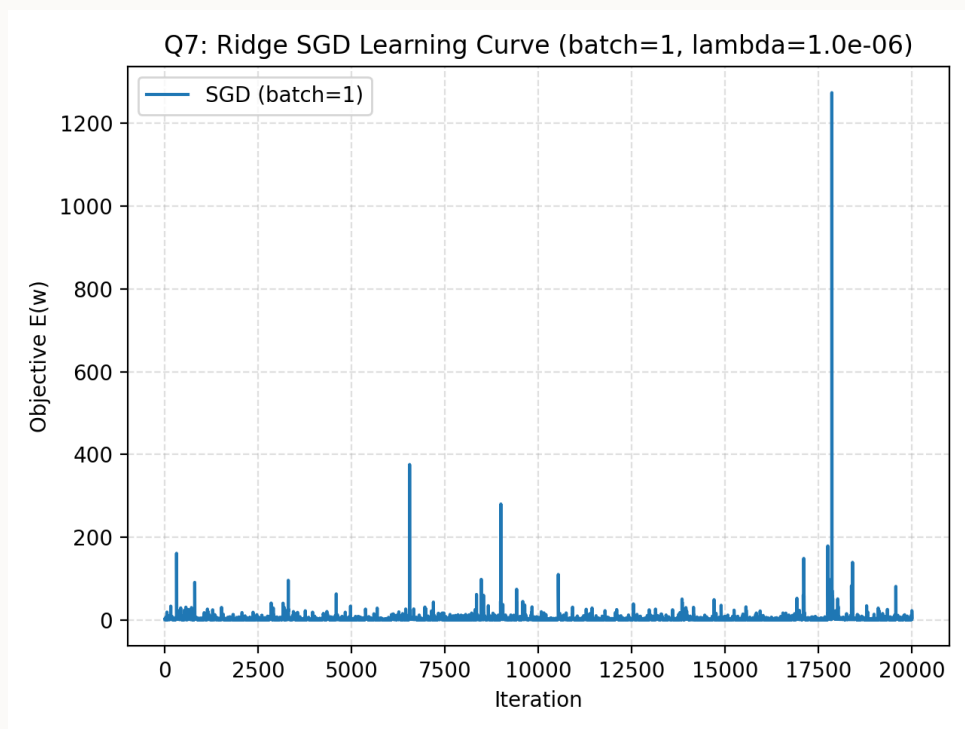


Figure 6: ridge_sgd_batch1

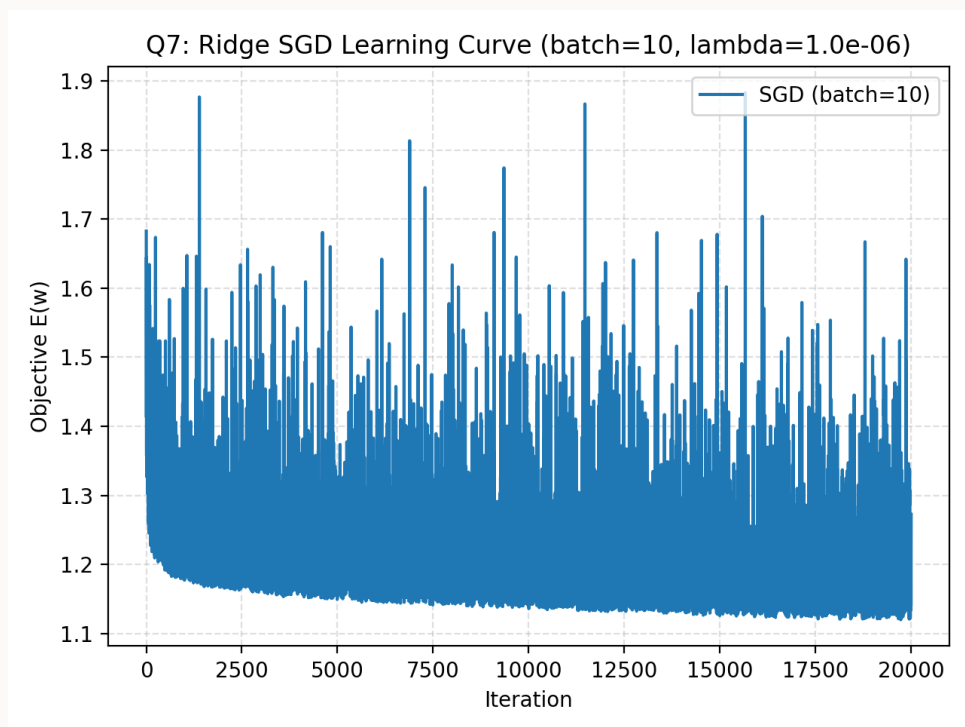


Figure 7: ridge_sgd_batch10

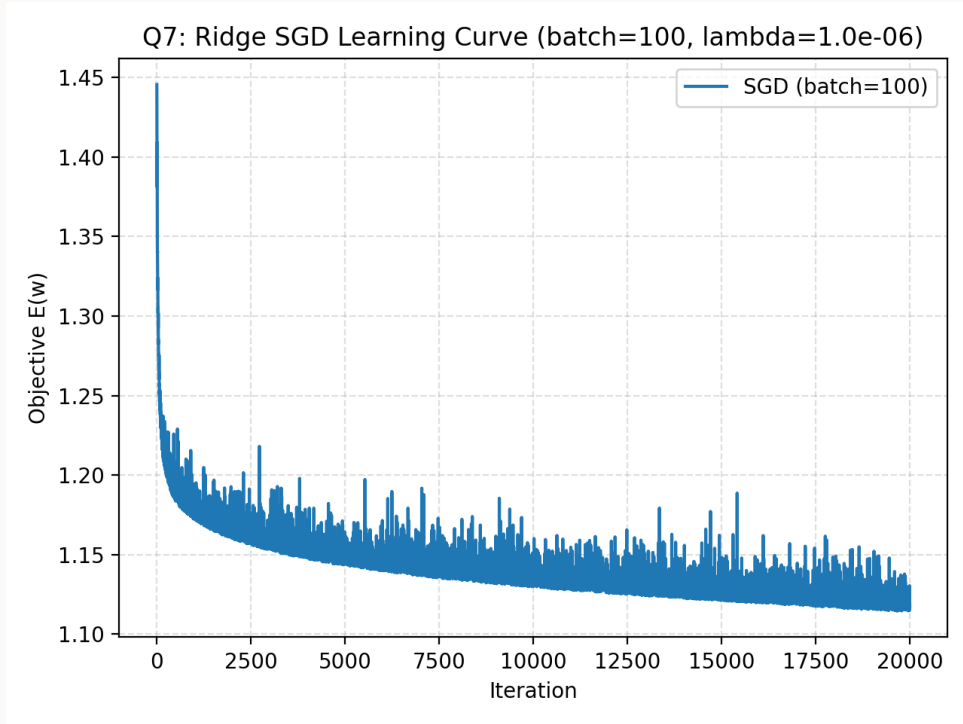


Figure 8: ridge_sgd_batch100

Assignment 8 (Bonus 12 points) Consider the hat matrix $\mathbf{H} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$, where $\mathbf{X}$ is an N by $d + 1$ matrix, and $\mathbf{X}^T \mathbf{X}$ is invertible.

- (a) (3 points) Show that $\mathbf{H}$ is symmetric.
- (b) (3 points) Show that $\mathbf{H}^K = \mathbf{H}$ for any positive integer K .
- (c) (3 points) If $\mathbf{I}$ is the identity matrix of size N , show that $(\mathbf{I} - \mathbf{H})^K = \mathbf{I} - \mathbf{H}$ for any positive integer K .
- (d) (3 points) Show that $\text{trace}(\mathbf{H}) = d + 1$, where the trace is the sum of diagonal elements.
[Hint: $\text{trace}(\mathbf{AB}) = \text{trace}(\mathbf{BA})$ for any matrices A, B of compatible sizes]

Proof. Let $H = X(X^T X)^{-1} X^T$, with $X^T X$ invertible.

- (a) H is symmetric

$$H^T = (X(X^T X)^{-1} X^T)^T = X((X^T X)^{-1})^T X^T.$$

Since $X^T X$ is symmetric, $(X^T X)^{-1}$ is also symmetric, so $((X^T X)^{-1})^T = (X^T X)^{-1}$. Hence $H^T = H$.

(b) $H^K = H$ for any positive integer K First show idempotence:

$$\begin{aligned}
 H^2 &= X(X^\top X)^{-1}X^\top X(X^\top X)^{-1}X^\top \\
 &= X(X^\top X)^{-1}(X^\top X)(X^\top X)^{-1}X^\top \\
 &= X(X^\top X)^{-1}X^\top \\
 &= H.
 \end{aligned}$$

Therefore $H^K = H$ for any integer $K \geq 1$ (by induction).

(c) $(I - H)^K = I - H$ for any positive integer K Using $H^2 = H$:

$$(I - H)^2 = I - 2H + H^2 = I - 2H + H = I - H,$$

so $(I - H)$ is also idempotent. Hence $(I - H)^K = I - H$ for any integer $K \geq 1$.

(d) $\text{trace}(H) = d + 1$ Using $\text{trace}(AB) = \text{trace}(BA)$ for compatible matrices:

$$\text{trace}(H) = \text{trace}(X(X^\top X)^{-1}X^\top) = \text{trace}((X^\top X)^{-1}X^\top X) = \text{trace}(I_{d+1}) = d + 1.$$